Preparing input data for LocusPick

Contents

The easy way: tool outputs as they are . . . 1
1. Signal tracks (bigWig) . . . 4
2. Peaks . . . 6
3. Gene models . . . 8
4. Differential expression tables . . . 9
5. Expression for insets . . . 10
6. Mappability . . . 10
7. Check everything together . . . 11
Suggested end-to-end workflow . . . 11

LocusPick needs six inputs. In most cases you can pass your tools' output **as it is**; the first section shows how. The numbered sections then give alternatives for each input, for when you want more control or your files come from elsewhere. Chunks that run on the bundled example data are executed when the vignette is built; chunks that need external tools (shown as `bash`) or your own files are shown but not run.

Input	Format LocusPick reads	Required?
Signal tracks	bigWig or BAM (one or several per condition), `GRanges`, chromDict, any function	for signal ranking and plots
Peaks	MACS2, SEACR, HOMER, DiffBind, nf-core, BED, `GRanges`	yes
Gene models	GTF/GFF, or `GRanges` with `type` and `gene_name`	yes
DE results (×2)	DESeq2 / edgeR / limma objects or tables, any CSV/TSV	yes
Expression for insets	`DESeqDataSet`, `DGEList`, `SummarizedExperiment`, matrix	optional
Mappability	bigWig (Umap)	optional, recommended

Golden rule: every input must use the **same genome build**. Run `checkInputs()` (last section) before anything else; it catches build and naming mismatches that otherwise fail silently.

The easy way: tool outputs as they are

Tool output	Function	Notes
DESeq2 `results()` object or saved table	`readDE()`	contrast read from the object and printed
edgeR `glmQLFTest()` / `exactTest()` / `topTags()`	`readDE()`	contrast read from the object and printed
limma fit (`eBayes()`) or `topTable()`	`readDE(fit, coef = ...)`	
Ensembl or Entrez gene IDs	`readDE(..., orgDb = )`	converted to symbols

Tool output	Function	Notes
MACS2/MACS3 `narrowPeak`, `broadPeak`, `_peaks.xls`; Genrich	`readPeaks()`	
SEACR `.stringent.bed` / `.relaxed.bed`	`readPeaks()`	
HOMER `peaks.txt`, `annotatePeaks.pl` output	`readPeaks()`	HOMER's gene column is kept for checking
DiffBind `dba.report()`	`readPeaks()`	fold change and FDR kept
Several replicate peak files	`readPeaks(c(...))`	consensus peaks (in 2 files by default)
nf-core cutandrun / chipseq / atacseq results folder	`findPipelineOutputs()`	lists bigWigs and peaks; flags IgG/input
Replicate bigWigs or BAMs	`signal = list(young = c("y1.bw", "y2.bw"))`	averaged on the fly
`DESeqDataSet`, `DGEList`	`expressionFrom()`	normalized counts / CPM for insets

`locusPick()` calls these for you, so a full run can be one call (see `vignette("LocusPick")`). For single-cell data (Seurat/Signac, `SingleCellExperiment`, Cell Ranger folders) see `vignette("single-cell")`. Every reader prints what it detected, so you can confirm it.

Differential expression

```
# a small simulated experiment
set.seed(3)
counts <- matrix(rnbinom(600, mu = 100, size = 10), nrow = 100,
                 dimnames = list(paste0("gene", 1:100), paste0("s", 1:6)))
counts[1:10, 4:6] <- counts[1:10, 4:6] * 6L
samples <- data.frame(genotype = factor(rep(c("WT", "KO"), each = 3),
                                        levels = c("WT", "KO")))
dds <- DESeq2::DESeq(DESeq2::DESeqDataSetFromMatrix(counts, samples, ~ genotype),
                     quiet = TRUE)
#> converting counts to integer mode
res <- DESeq2::results(dds, contrast = c("genotype", "KO", "WT"))

ko <- readDE(res)
#> res: DESeq2 results | lfc = 'log2FoldChange', padj = 'padj', gene = row names
#>   (symbol IDs) | 100 genes
#>   contrast: genotype KO vs WT
head(ko, 3)
#>     gene      lfc         padj
#> 1  gene3 3.057907 4.112381e-23
#> 2  gene5 3.081779 6.328557e-19
#> 3 gene10 2.834081 1.444859e-17
```

The printed contrast ("genotype KO vs WT") confirms the direction. Saved tables work the same way, but a plain file does not record its contrast, so state it:

```
csv <- file.path(td, "ko_vs_wt.csv")
write.csv(as.data.frame(res), csv)
ko <- readDE(csv, direction = "KO/WT")
#> ko_vs_wt.csv: DESeq2-style table | lfc = 'log2FoldChange', padj = 'padj',
#>   gene = 'rownames_col' (symbol IDs) | 100 genes
```

```
#>   contrast (stated): KO/WT
```

edgeR and limma objects are read directly too:

```
y <- edgeR::normLibSizes(edgeR::DGEList(counts, group = samples$genotype))
design <- model.matrix(~ samples$genotype)
qlf <- edgeR::glmQLFTest(edgeR::glmQLFit(edgeR::estimateDisp(y, design), design), coef = 2)
ko_edger <- readDE(qlf)
#> qlf: edgeR DGELRT | lfc = 'logFC', padj = 'FDR', gene = row names (symbol
#>   IDs) | 100 genes
#>   contrast: samples$genotypeKO

fit <- limma::eBayes(limma::lmFit(limma::voom(y, design), design))
ko_limma <- readDE(fit, coef = 2, direction = "KO/WT")
#> fit: limma fit | lfc = 'logFC', padj = 'adj.P.Val', gene = row names (symbol
#>   IDs) | 100 genes
#>   contrast: samples$genotypeKO  (stated: KO/WT)
```

Peaks

```
np <- file.path(td, "young_rep1_peaks.narrowPeak")
writeLines(c("chr1\t4799\t5300\tp1\t100\t.\t5.2\t12.1\t9.8\t251",
             "chr1\t19799\t20300\tp2\t80\t.\t4.1\t10.0\t8.0\t200"), np)
seacr <- file.path(td, "young_rep2.stringent.bed")
writeLines(c("chr1\t4850\t5250\t120.5\t3.2\tchr1:5000-5100",
             "chr1\t40000\t40400\t80.1\t2.1\tchr1:40100-40200"), seacr)

readPeaks(np)
#> young_rep1_peaks.narrowPeak: narrowPeak, 2 peaks
#> GRanges object with 2 ranges and 8 metadata columns:
#>       seqnames      ranges strand |        name     score    signal    pvalue
#>          <Rle>   <IRanges>  <Rle> | <character> <integer> <numeric> <numeric>
#>   [1]     chr1   4800-5300      * |          p1       100       5.2      12.1
#>   [2]     chr1 19800-20300      * |          p2        80       4.1      10.0
#>          qvalue    summit peak_format          peak_id
#>       <numeric> <integer> <character>      <character>
#>   [1]       9.8      5051  narrowPeak   chr1:4800-5300
#>   [2]       8.0     20000  narrowPeak chr1:19800-20300
#>   -------
#>   seqinfo: 1 sequence from an unspecified genome; no seqlengths
readPeaks(c(np, seacr))     # consensus across replicates (and callers)
#> young_rep1_peaks.narrowPeak: narrowPeak, 2 peaks
#> young_rep2.stringent.bed: seacr, 2 peaks
#> Consensus: 1 peaks present in >= 2 of 2 replicates
#> GRanges object with 1 range and 3 metadata columns:
#>       seqnames    ranges strand |    n_reps            peak_format
#>          <Rle> <IRanges>  <Rle> | <integer>            <character>
#>   [1]     chr1 4800-5300      * |         2 consensus (narrowPea..
#>              peak_id
#>          <character>
#>   [1] chr1:4800-5300
#>   -------
#>   seqinfo: 1 sequence from an unspecified genome; no seqlengths
```

Pipeline output folders

```
# a mock nf-core/cutandrun results folder
nf <- file.path(td, "results")
for (d in c("03_peak_calling/03_bed_to_bigwig", "03_peak_calling/04_called_peaks/seacr"))
    dir.create(file.path(nf, d), recursive = TRUE)
invisible(file.copy(system.file("extdata", "toy_A.bw", package = "LocusPick"),
          file.path(nf, "03_peak_calling/03_bed_to_bigwig", c("Y_R1.bigWig", "Y_R2.bigWig"))))
invisible(file.copy(system.file("extdata", "toy_B.bw", package = "LocusPick"),
          file.path(nf, "03_peak_calling/03_bed_to_bigwig", c("O_R1.bigWig", "IgG_R1.bigWig"))))
invisible(file.copy(seacr, file.path(nf, "03_peak_calling/04_called_peaks/seacr",
                           "Y_R1.seacr.peaks.stringent.bed")))

out <- findPipelineOutputs(nf)
out[, c("type", "caller", "sample", "control")]
#>     type caller sample control
#> 1 bigwig   <NA> IgG_R1    TRUE
#> 2 bigwig   <NA>   O_R1   FALSE
#> 3 bigwig   <NA>   Y_R1   FALSE
#> 4 bigwig   <NA>   Y_R2   FALSE
#> 5  peaks  seacr   Y_R1   FALSE

signal <- signalFromFiles(out, c(young = "^Y_", old = "^O_"))   # IgG excluded
#> young: Y_R1.bigWig, Y_R2.bigWig
#> old: O_R1.bigWig
```

Expression for insets

```
ins <- expressionFrom(dds, group = "genotype", panel = "Knockout")
head(ins, 3)
#>    gene    panel group     value
#> 1 gene1 Knockout    WT  65.64254
#> 2 gene2 Knockout    WT  67.72643
#> 3 gene3 Knockout    WT 100.02673
```

BAM files

When bigWigs are not available yet, BAM files can be used directly; coverage is computed (CPM, duplicates and MAPQ < 10 removed) only where needed:

```
bam <- system.file("extdata", "toy.bam", package = "LocusPick")
signalAtPeaks(ex$peaks[1:3], list(bam = bamSource(bam)))
#> BAM signal for toy.bam: paired-end, CPM, MAPQ >= 10, duplicates removed. For figures, bigWigs from d
#>            peak_id    bam_mean bam_cov
#> 1   chr1:4800-5300 132820.5128       1
#> 2 chr1:19800-20300  68579.7635       1
#> 3 chr1:27000-27400    575.4844       1
```

For figures, bigWigs made with deepTools (section 1) give more control and are much faster.

1. Signal tracks (bigWig)

One bigWig per **condition** is easiest to read in figures; one per **replicate** lets reviewers see consistency. You can make both.

Option A: deepTools from BAM (recommended)

```
# per replicate; CPM-normalized, 20 bp bins, duplicates removed
bamCoverage -b young_rep1.bam -o young_rep1.bw \
    --normalizeUsing CPM --binSize 20 --ignoreDuplicates \
    --extendReads -p 8            # --extendReads: paired-end fragments

# condition mean across replicates (deepTools >= 3.5.2)
bigwigAverage -b young_rep1.bw young_rep2.bw young_rep3.bw \
    -o young_mean.bw --binSize 20 -p 8
```

Use the same normalization and bin size for every sample you will compare. `--effectiveGenomeSize` is only needed with `--normalizeUsing RPGC`.

Option B: from BAM in R

No command-line tools needed; uses Bioconductor packages only.

```
library(GenomicAlignments)
bamToBigWig <- function(bam, out, paired = TRUE) {
    aln <- if (paired) granges(readGAlignmentPairs(bam), on.discordant.seqnames = "drop")
           else granges(readGAlignments(bam))
    cov <- coverage(aln) * (1e6 / length(aln))       # CPM
    rtracklayer::export(cov, out, format = "bigWig")
}
bamToBigWig("young_rep1.bam", "young_rep1.bw")
```

To average replicates in R, add coverages and divide:

```
covs <- lapply(c("young_rep1.bw", "young_rep2.bw"), rtracklayer::import, as = "RleList")
rtracklayer::export(Reduce(`+`, covs) / length(covs), "young_mean.bw", format = "bigWig")
```

Option C: from bedGraph (MACS2, SEACR, nf-core)

MACS2 writes a fragment pileup with `-B`; add `--SPMR` for signal per million reads. SEACR and many CUT&RUN pipelines also produce bedGraphs.

```
macs2 callpeak -t young_rep1.bam -c igg.bam -f BAMPE -g mm \
    -n young_rep1 -B --SPMR --outdir macs2/
# -> macs2/young_rep1_treat_pileup.bdg

# bedGraph -> bigWig (UCSC tool; chrom.sizes must match your genome build)
LC_ALL=C sort -k1,1 -k2,2n young_rep1_treat_pileup.bdg > tmp.sorted.bdg
bedGraphToBigWig tmp.sorted.bdg mm10.chrom.sizes young_rep1.bw
```

Or in R (works for any bedGraph that fits in memory):

```
# demo: write a bedGraph from the example track, then convert it back
bg <- file.path(td, "demo.bedGraph")
rtracklayer::export(fetchSignal(ex$signal$A, GRanges("chr1:1-60000")), bg,
                    format = "bedGraph")

gr <- rtracklayer::import(bg, format = "bedGraph")
seqlengths(gr) <- c(chr1 = 60000)          # use your genome's chrom.sizes
rtracklayer::export(gr, file.path(td, "demo.bw"), format = "bigWig")
signalAtPeaks(ex$peaks[1:2], list(demo = file.path(td, "demo.bw")))
#>            peak_id demo_mean demo_cov
```

```
#> 1   chr1:4800-5300 0.6069934         1
#> 2 chr1:19800-20300 0.4915202         1
```

A bedGraph can also be used directly, without converting, with `rangesSource(rtracklayer::import(bg))`.

Option D: existing chromDicts or other formats

```
signal <- list(
    CNR_young = chromDictSource(readRDS("Yng.rds"), binSize = 20),
    ATAC      = functionSource(function(chr, start, end) myReader(chr, start, end)))
```

`functionSource()` accepts any function returning `start`/`end` (or `pos`) and `score` (or `value`), so other storage formats need no conversion.

Normalization and spike-ins

CPM/BPM assumes the total amount of binding is the same in every condition. If a factor binds less *everywhere* in one condition, CPM hides or distorts that. Conversely, a library-size artifact can look like global loss. For CUT&RUN with *E. coli* or spike-in DNA, scale by the spike-in instead:

```
# scale = 1 / (spike-in reads in millions); compute per sample
bamCoverage -b old_rep1.bam -o old_rep1.spike.bw --binSize 20 \
    --scaleFactor 0.42 --ignoreDuplicates --extendReads
```

Whichever you choose, `signalShift()` will tell you whether a change at your examples differs from the change at all peaks.

2. Peaks

Option A: MACS2 narrowPeak / broadPeak / xls

```
pk <- readPeaks("macs2/young_rep1_peaks.narrowPeak")   # or _peaks.xls
```

Option B: SEACR

SEACR writes `chr start end total_signal max_signal max_region`; `readPeaks()` recognizes it and keeps the signal columns:

```
pk <- readPeaks("young_rep1.stringent.bed")
```

Option C: HOMER

`readPeaks()` reads `peaks.txt` and `annotatePeaks.pl` output directly (HOMER coordinates are 1-based; they are kept as is):

```
pk <- readPeaks("annotated_peaks.txt")
```

From `annotatePeaks.pl`, the gene column is kept as `homer_gene`. To use HOMER's assignment rather than LocusPick's, pass `geneCol = "homer_gene"`; it is then checked against your gene models, since annotations can link enhancer peaks to genes tens of kilobases away.

Option D: DiffBind

```
db <- DiffBind::dba.report(dba_obj, th = 1)       # all sites
pk <- readPeaks(db)                                # keeps diff_fold, diff_fdr
```

Differential binding statistics can then be used in the ranking, e.g. `rankLoci(cand, c(diff_fold = 1, abs_lfc_perturbation = 1))`.

Option E: data.frame from any tool

```
df <- data.frame(chr = "chr1", start = c(4799, 19799), end = c(5300, 20300))
asPeaks(df, zeroBased = TRUE)    # TRUE if the starts came from a BED file
#> GRanges object with 2 ranges and 1 metadata column:
#>       seqnames      ranges strand |          peak_id
#>          <Rle>   <IRanges>  <Rle> |      <character>
#>   [1]     chr1   4800-5300      * |   chr1:4800-5300
#>   [2]     chr1 19800-20300      * | chr1:19800-20300
#>   -------
#>   seqinfo: 1 sequence from an unspecified genome; no seqlengths
```

Consensus peaks across replicates

Peaks found in at least k replicates are more reliable examples than peaks from a single sample. `readPeaks()` does this when given several files:

```
pk <- readPeaks(c("rep1_peaks.narrowPeak", "rep2_peaks.narrowPeak",
                  "rep3_peaks.narrowPeak"), minReps = 2,
                blacklist = "mm10-blacklist.v2.bed.gz")
```

The same by hand, in R:

```
consensusPeaks <- function(reps, minReps = 2) {
    reps <- lapply(reps, function(x) granges(asPeaks(x)))
    merged <- reduce(do.call(c, unname(reps)))
    n <- Reduce(`+`, lapply(reps, function(r) overlapsAny(merged, r)))
    merged[n >= minReps]
}
# demo: three "replicates" made by jittering the example peaks
set.seed(1)
reps <- lapply(1:3, function(i) {
    p <- granges(ex$peaks)[sample(length(ex$peaks), 6)]
    shift(p, sample(-50:50, length(p), replace = TRUE))
})
cons <- consensusPeaks(reps, minReps = 2)
length(cons)
#> [1] 6
```

With bedtools:

```
cat rep1.bed rep2.bed rep3.bed | cut -f1-3 | sort -k1,1 -k2,2n \
    | bedtools merge > merged.bed
bedtools intersect -a merged.bed -b rep1.bed rep2.bed rep3.bed -c \
    | awk '$4 >= 2' | cut -f1-3 > consensus.bed
```

Condition-specific sets follow the same pattern:

```
young <- cons
old   <- granges(ex$peaks)[c(1, 2, 7)]
shared     <- young[overlapsAny(young, old)]
youngOnly  <- young[!overlapsAny(young, old)]
c(shared = length(shared), youngOnly = length(youngOnly))
```

```
#>    shared youngOnly
#>         2         4
```

A peak-set label can be carried into the candidate table as a column:

```
pk <- asPeaks(young)
pk$peak_set <- ifelse(overlapsAny(pk, old), "shared", "young-only")
```

Blacklisted regions

Remove ENCODE blacklist regions (Amemiya et al. 2019; `mm10-blacklist.v2`, `hg38-blacklist.v2` from the Boyle lab Blacklist repository) before ranking. These regions give high signal in every assay and make misleading examples.

```
bl <- rtracklayer::import("mm10-blacklist.v2.bed.gz")
pk <- pk[!overlapsAny(pk, bl)]
```

3. Gene models

Option A: GTF from Ensembl, GENCODE or UCSC

```
genes <- readGeneModels(system.file("extdata", "toy.gtf", package = "LocusPick"))
genes
#> locusGeneModels: 6 genes, 18 collapsed exons
```

Use the GTF that matches the genome build of your alignments: for example GENCODE vM25 or Ensembl 102 for mm10/GRCm38. Ensembl names chromosomes `1`, `2`, …; UCSC and GENCODE use `chr1`. LocusPick converts between the two automatically, but `checkInputs()` reports it so you notice.

Option B: TxDb + OrgDb (no GTF file)

```
library(TxDb.Mmusculus.UCSC.mm10.knownGene)
library(org.Mm.eg.db)
txdb <- TxDb.Mmusculus.UCSC.mm10.knownGene
g <- GenomicFeatures::genes(txdb)
e <- unlist(GenomicFeatures::exonsBy(txdb, by = "gene"))
e$gene_id <- names(e)
sym <- AnnotationDbi::mapIds(org.Mm.eg.db, c(g$gene_id, e$gene_id),
                             "SYMBOL", "ENTREZID")
g$type <- "gene"; g$gene_name <- sym[g$gene_id]
e$type <- "exon"; e$gene_name <- sym[e$gene_id]
mcols(e) <- mcols(e)[, c("gene_id", "type", "gene_name")]
genes <- readGeneModels(c(g, e))
```

Option C: any GRanges

`readGeneModels()` accepts a `GRanges` with a `type` column (`gene`, `transcript` or `exon`) and a gene name column:

```
gr <- GRanges("chr1", IRanges(c(100, 100, 800), c(2000, 300, 2000)),
              strand = "+", type = c("gene", "exon", "exon"),
              gene_name = "myGene")
readGeneModels(gr)$genes
#> GRanges object with 1 range and 2 metadata columns:
#>       seqnames    ranges strand |   gene_name       tss
```

```
#>          <Rle> <IRanges>  <Rle> | <character> <integer>
#>   [1]     chr1  100-2000      + |      myGene       100
#>   -------
#>   seqinfo: 1 sequence from an unspecified genome; no seqlengths
```

Harmonizing symbols

If the DE tables, peak annotation and GTF come from different annotation releases, map everything to current symbols with the same function:

```
h <- function(s) harmonizeSymbols(s, org.Mm.eg.db::org.Mm.eg.db)
genes <- readGeneModels("genes.gtf", harmonize = h)
```

4. Differential expression tables

`defineGroups()` needs two results: the **perturbation** (e.g. knockout vs control) and the **condition** (e.g. old vs young). `readDE()` reads them from DESeq2, edgeR or limma (objects or files) and any CSV/TSV with recognizable columns; `defineGroups()` calls it automatically.

> **Check the direction of every contrast.** Fold changes must be *perturbation / control* and *condition / baseline* (e.g. KO / WT, old / young). A reversed contrast swaps activated and repressed targets without any error. `readDE()` prints the contrast recorded in DESeq2 and edgeR objects; for plain tables, state it with `direction`.

Option A: pass the results directly

```
ko  <- readDE(DESeq2::results(dds, contrast = c("genotype", "KO", "WT")))
age <- readDE(edgeR::glmQLFTest(fit, coef = "ageold"))
ko  <- readDE(limma_fit, coef = "genotypeKO")
```

Option B: saved tables with unusual column names

Name the columns when detection fails:

```
odd <- data.frame(symbol = c("geneA", "geneB"), effect = c(-2, 1.5),
                  q = c(1e-6, 1e-5))
readDE(odd, gene = "symbol", lfc = "effect", padj = "q", direction = "KO/WT")
#> odd: table | lfc = 'effect', padj = 'q', gene = 'symbol' (symbol IDs) | 2
#>   genes
#>   contrast (stated): KO/WT
#>    gene  lfc  padj
#> 1 geneA -2.0 1e-06
#> 2 geneB  1.5 1e-05
```

Option C: Ensembl or Entrez IDs

```
ko <- readDE(res, orgDb = org.Mm.eg.db::org.Mm.eg.db)    # ENSMUSG... -> symbols
```

Version suffixes (`ENSMUSG00000017950.16`) are removed first. When several IDs map to one symbol, the most significant row is kept.

Restricting to bound genes

To restrict groups to genes with a peak (as "direct targets"), pass them as `targets`:

```
bound <- unique(assignPeaks(ex$peaks, ex$genes)$gene)
de <- read.delim(system.file("extdata", "toy_de.tsv", package = "LocusPick"))
defineGroups(deTable(de, lfc = "ko_lfc", padj = "ko_padj"),
             deTable(de, lfc = "cond_lfc", padj = "cond_padj"),
             targets = bound, conditionName = "age")[, c("gene", "group")]
#>    gene              group
#> 1 geneA activated_age_down
#> 2 geneE activated_age_down
#> 3 geneC   activated_age_up
#> 4 geneD repressed_age_down
#> 5 geneB   repressed_age_up
```

5. Expression for insets

`plotLocus()` draws expression from a long table with one row per sample: `gene`, `panel` (one facet per experiment), `group`, `value`.

Option A: from DESeq2, edgeR or SummarizedExperiment objects

```
ko_expr  <- expressionFrom(dds_ko,  group = "genotype", panel = "Knockout",
                           levels = c("WT", "KO"))
age_expr <- expressionFrom(y_age,   group = "group",    panel = "Aging")   # DGEList: CPM
expression <- rbind(ko_expr, age_expr)
```

Option B: from any matrix and a sample vector

TPM, normalized counts, or any gene × sample matrix:

```
mat <- matrix(rpois(18, 100), nrow = 3,
              dimnames = list(c("geneA", "geneB", "geneC"), paste0("s", 1:6)))
long <- expressionFrom(mat, group = rep(c("Young", "Old"), each = 3), panel = "Aging")
head(long)
#>    gene panel group value
#> 1 geneA Aging Young   103
#> 2 geneB Aging Young   101
#> 3 geneC Aging Young   108
#> 4 geneA Aging Young    99
#> 5 geneB Aging Young   105
#> 6 geneC Aging Young   110
```

Groups are drawn in the order given by `levels`, or in the order they first appear; put the control first. Pass `orgDb` to convert Ensembl row names.

6. Mappability

Option A: Umap multi-read mappability (recommended)

Umap (Karimzadeh et al. 2018) provides, for each base, the fraction of overlapping k-mers that map uniquely. The authors distribute mm9, mm10, hg19 and hg38 as bedGraphs; the bundled script converts one to a bigWig once:

```
system.file("scripts", "make_umap_bw.sh", package = "LocusPick")
#> [1] "/usr/local/lib/R/site-library/LocusPick/scripts/make_umap_bw.sh"
```

```
bash make_umap_bw.sh 50     # k: closest to your read length (24, 36, 50, 100)
```

Run it on a node with ~16 GB memory; the intermediate bedGraph is several GB.

Option B: blacklist only

If you cannot use Umap, remove blacklisted regions from the peaks (section 2) and leave `mappability = NULL`. This removes the worst regions but not paralog clusters.

Option C: build Umap for another genome

The Umap software (`bioconda::umap`) computes mappability for any genome; see its documentation. It needs substantial compute (the authors report about 200 CPU hours per read length).

7. Check everything together

```
checkInputs(ex$peaks, ex$genes, signal = ex$signal, groups = ex$groups)
#> [ok  ] chromosome naming: peaks / genes      UCSC / UCSC
#> [ok  ] peak chromosomes found in gene models 100%
#> [ok  ] peaks within 5,000 bp of a gene       100%
#> [warn] signal 'A': peaks with data           88%
#> [warn] signal 'B': peaks with data           88%
#> [ok  ] grouped genes found in gene models    100%
#>         Peaks without signal: check build/naming of the track, or gaps in its coverage.
```

The warning is expected here: the example data deliberately contain one peak in a gap of the signal tracks. On real data, a handful of such peaks is normal; a large fraction points to a different genome build or chromosome naming in the track.

A common failure, gene models from a different build than the peaks, looks like this:

```
shifted <- shift(ex$genes$genes, 1e6)   # simulate gene models from another build
bad <- ex$genes; bad$genes <- shifted
checkInputs(ex$peaks, bad)
#> [ok  ] chromosome naming: peaks / genes      UCSC / UCSC
#> [ok  ] peak chromosomes found in gene models 100%
#> [fail] peaks within 5,000 bp of a gene       0%
#>         Few peaks are near genes: are peaks and gene models from the same genome build?
```

Suggested end-to-end workflow

```
# 1. signal: one bigWig per replicate and per condition
for s in young_rep{1..3} old_rep{1..3}; do
  bamCoverage -b $s.bam -o $s.bw --normalizeUsing CPM --binSize 20 \
      --ignoreDuplicates --extendReads -p 8
done
bigwigAverage -b young_rep*.bw -o young.bw --binSize 20
bigwigAverage -b old_rep*.bw   -o old.bw   --binSize 20

# 2. mappability (once per genome and read length)
bash make_umap_bw.sh 50
```

```
library(LocusPick)
library(org.Mm.eg.db)
```

```
res <- locusPick(
    peaks        = c("young_rep1_peaks.narrowPeak", "young_rep2_peaks.narrowPeak",
                     "young_rep3_peaks.narrowPeak"),
    blacklist    = "mm10-blacklist.v2.bed.gz",
    perturbation = DESeq2::results(dds_ko, contrast = c("genotype", "KO", "WT")),
    condition    = DESeq2::results(dds_age, contrast = c("age", "old", "young")),
    genes        = "gencode.vM25.annotation.gtf",
    orgDb        = org.Mm.eg.db,
    signal       = list(CNR_young = "young.bw", CNR_old = "old.bw"),
    mappability  = "k50.Umap.MultiTrackMappability.bw",
    expression   = list(expressionFrom(dds_ko,  "genotype", panel = "Knockout"),
                        expressionFrom(dds_age, "age",      panel = "Aging")),
    conditionName = "age")
res
reportLoci(res, "review.pdf")
```

To adjust the ranking or plots, continue step by step as described in `vignette("LocusPick")`.